

Design Patterns Structurels en PHP

Objectif des patterns structurels

Les **patterns structurels** décrivent des **manières d'organiser et de combiner des classes ou objets**, pour former des structures plus grandes et plus flexibles.

Ils facilitent la **composition**, l'**adaptation**, et la **réutilisation** des objets et interfaces.

Sommaire :

1. [Adapter](#)
 2. [Decorator](#)
 3. [Composite](#)
 4. [Facade](#)
 5. [Proxy](#)
 6. [Bridge](#)
 7. [Flyweight](#)
-

1. Adapter

 But :

Permet à deux interfaces incompatibles de travailler ensemble. Il **convertit** l'interface d'une classe en une autre.

 Exemple en PHP :

```
// Interface attendue
interface LecteurAudio {
    public function jouer(string $fichier);
}

// Classe existante incompatible
class LecteurMP3 {
    public function lireFichier($nom) {
        echo "Lecture MP3 : $nom";
    }
}

// Adaptateur
class MP3Adapter implements LecteurAudio {
    private $lecteur;

    public function __construct(LecteurMP3 $l) {
        $this->lecteur = $l;
    }
}
```

```

    }

    public function jouer(string $fichier) {
        $this->lecteur->lireFichier($fichier);
    }
}

// Utilisation
$lecteur = new MP3Adapter(new LecteurMP3());
$lecteur->jouer("musique.mp3");

```

2. Decorator

 But :

Ajouter dynamiquement des **comportements supplémentaires** à un objet **sans modifier sa classe**.

 Exemple en PHP :

```

interface Message {
    public function afficher(): string;
}

class MessageSimple implements Message {
    public function afficher(): string {
        return "Bonjour";
    }
}

class Encadreur implements Message {
    protected $message;

    public function __construct(Message $m) {
        $this->message = $m;
    }

    public function afficher(): string {
        return "[" . $this->message->afficher() . "]";
    }
}

class Majusculeur implements Message {
    protected $message;

    public function __construct(Message $m) {
        $this->message = $m;
    }

    public function afficher(): string {
        return strtoupper($this->message->afficher());
    }
}

```

```

    }
}

// Composition
$msg = new Majusculeur(new Encadreur(new MessageSimple()));
echo $msg->afficher(); // [BONJOUR]

```

3. 🌲 Composite

📌 But :

Permet de **composer des objets en structures arborescentes**, et de manipuler uniformément **objets simples et composés**.

📦 Exemple en PHP :

```

interface Element {
    public function afficher(): string;
}

class Fichier implements Element {
    private $nom;

    public function __construct($nom) {
        $this->nom = $nom;
    }

    public function afficher(): string {
        return $this->nom;
    }
}

class Dossier implements Element {
    private $nom;
    private $contenu = [];

    public function __construct($nom) {
        $this->nom = $nom;
    }

    public function ajouter(Element $e) {
        $this->contenu[] = $e;
    }

    public function afficher(): string {
        $res = "[".$this->nom."";
        foreach ($this->contenu as $e) {
            $res .= "\n - " . $e->afficher();
        }
        return $res;
    }
}

```

```

    }
}

// Utilisation
$dossier = new Dossier("Documents");
$dossier->ajouter(new Fichier("cv.pdf"));
$dossier->ajouter(new Fichier("photo.jpg"));

echo $dossier->afficher();

```

4. 🤖 Facade

📌 But :

Fournit une **interface simplifiée** à un **ensemble complexe de classes** ou sous-systèmes.

📦 Exemple en PHP :

```

class CPU {
    public function demarrer() { echo "CPU démarré\n"; }
}
class RAM {
    public function charger() { echo "RAM chargée\n"; }
}
class Disque {
    public function lire() { echo "Disque lu\n"; }
}

// Facade
class Ordinateur {
    private $cpu;
    private $ram;
    private $disque;

    public function __construct() {
        $this->cpu = new CPU();
        $this->ram = new RAM();
        $this->disque = new Disque();
    }

    public function demarrerPC() {
        $this->cpu->demarrer();
        $this->ram->charger();
        $this->disque->lire();
    }
}

// Utilisation
$pc = new Ordinateur();
$pc->demarrerPC();

```

5. 🕵️ Proxy

📌 But :

Fournir un **représentant** d'un objet pour **contrôler son accès** (lazy loading, cache, sécurité...).

📦 Exemple en PHP :

```
interface Document {
    public function afficher();
}

class FichierPDF implements Document {
    private $nom;

    public function __construct($nom) {
        $this->nom = $nom;
        echo "Chargement fichier $nom...\n";
    }

    public function afficher() {
        echo "Affichage de $this->nom\n";
    }
}

// Proxy
class ProxyPDF implements Document {
    private $nom;
    private $document = null;

    public function __construct($nom) {
        $this->nom = $nom;
    }

    public function afficher() {
        if ($this->document === null) {
            $this->document = new FichierPDF($this->nom);
        }
        $this->document->afficher();
    }
}

// Utilisation
$pdf = new ProxyPDF("rapport.pdf");
$pdf->afficher(); // Charge puis affiche
$pdf->afficher(); // Affiche seulement (déjà chargé)
```

6. 🌉 Bridge

📌 But :

Séparer l'abstraction de son implémentation pour que les deux puissent évoluer **indépendamment**.

📦 Exemple en PHP :

```
interface Couleur {
    public function remplir();
}

class Rouge implements Couleur {
    public function remplir() { echo "Rempli de rouge"; }
}

class Bleu implements Couleur {
    public function remplir() { echo "Rempli de bleu"; }
}

abstract class Forme {
    protected $couleur;
    public function __construct(Couleur $c) {
        $this->couleur = $c;
    }

    abstract public function dessiner();
}

class Cercle extends Forme {
    public function dessiner() {
        echo "Cercle ";
        $this->couleur->remplir();
    }
}

// Utilisation
$f = new Cercle(new Rouge());
$f->dessiner(); // Cercle Rempli de rouge
```

7. 🪶 Flyweight

📌 But :

Réduire l'utilisation de mémoire en **partageant les objets similaires**.

📦 Exemple en PHP :

```

class Police {
    private $nom;
    public function __construct($nom) {
        $this->nom = $nom;
    }
    public function afficher($texte) {
        echo "$texte en $this->nom\n";
    }
}

class PoliceFactory {
    private static $polices = [];

    public static function getPolice($nom) {
        if (!isset(self::$polices[$nom])) {
            self::$polices[$nom] = new Police($nom);
        }
        return self::$polices[$nom];
    }
}

// Utilisation
$p1 = PoliceFactory::getPolice("Arial");
$p2 = PoliceFactory::getPolice("Arial");

$p1->afficher("Titre");
$p2->afficher("Sous-titre");

var_dump($p1 === $p2); // true

```